

Computational Lab 2

640:292 - Honors Track Calculus IV

April 9, 2025

Which programming language to use?

You may use any programming language you want, such as Matlab, Mathematica, Python, Maxima, etc... This lab requires basic programming, and graphing. Note that you will also need some subroutines and `for`-loops, which Mathematica does not handle very elegantly. Whatever you may choose, clearly indicate which programming language you have used for this assignment. Throughout this problem, report numerical values **in decimal form**.

Problem

In this lab, we will use the Picard iteration to compute approximate solutions to differential equations. Recall that the Picard iteration for the differential equation

$$\mathbf{x}'(t) = \mathbf{v}(\mathbf{x}(t), t), \quad \mathbf{x}(t_0) = \mathbf{x}_0 \quad (1)$$

is defined as

$$\mathbf{x}_{j+1}(t) = \mathbf{x}_0 + \int_{t_0}^t \mathbf{v}(\mathbf{x}_j(s), s) \, ds. \quad (2)$$

In class and in the homework, we have seen examples in which the integrals involved in the Picard iteration can be computed exactly in terms of elementary functions. However, this is not always the case, and in this lab we will discuss methods to compute the Picard iteration that do not require an exact evaluation of the integral. We will focus on the one-dimensional case, but everything discussed here easily extends to higher dimension.

We will approximate the integral by its Riemann sum. We fix an integer $N \in \mathbb{N}$ and approximate

$$\int_{t_0}^t v(x_j(s), s) \, ds \approx \frac{t - t_0}{N} \sum_{n=0}^N F(t_0 + (t - t_0) \frac{n}{N}) \quad (3)$$

with

$$F(t) := v(x_j(t), t). \quad (4)$$

By the Riemann theorem, provided v and x_j are continuous,

$$\lim_{N \rightarrow \infty} \frac{t - t_0}{N} \sum_{n=0}^N F(t_0 + (t - t_0) \frac{n}{N}) = \int_{t_0}^t v(x_j(s), s) \, ds \quad (5)$$

so the finite sum is an approximation for the integral. (This is actually not a very efficient way to approximate the integral, but there are other techniques that are similar, though more complicated, which work extremely well.)

Exercise 1

Write a routine `integrate_riemann`. This routine should take the following arguments: a function F , two numbers $t_0, t \in \mathbb{R}$ and an integer $N \in \mathbb{N}$. The routine then returns

$$\frac{t - t_0}{N} \sum_{n=0}^N F(t_0 + (t - t_0) \frac{n}{N}). \quad (6)$$

At this stage, it might seem that we can go ahead and compute Picard iterates, but as we shall now see, this will not work very well in practice.

Exercise 2

Write a routine `picard`. This routine should take the following arguments: two functions v and x_j , two numbers $t_0, t \in \mathbb{R}$, a number x_0 and an integer $N \in \mathbb{N}$. The routine then returns the Riemann sum approximation of

$$x_0 + \int_{t_0}^t v(x_j(s), s) \quad (7)$$

(use the routine `integrate_riemann` to carry out the integration).

Let's try this out on an example for which we know the answer: consider

$$x'(t) = \cos(t)x(t), \quad x(0) = 1 \quad (8)$$

whose solution is

$$x(t) = e^{\sin(t)}. \quad (9)$$

Exercise 3

Take $t_0 = 0$, $N = 100$. Write routines to compute $x_1(t)$, $x_2(t)$, $x_3(t)$ and $x_4(t)$ (if you are using Python, you might want to make use of the `lambda` keyword instead of defining new routines). (*Hint:* To compute $x_2(t)$, use the routine you wrote to compute $x_1(t)$; and similarly for x_3 and x_4 .) Compute $x_1(1)$, $x_2(1)$, $x_3(1)$ and $x_4(1)$. You will notice that the time it takes to compute $x_j(1)$ grows exponentially with j . (The time it takes to compute x_j goes like N^j .) In this way, it would take an extremely long time to compute $x_5(1)$.

The issue is that to compute $x_j(1)$, we need to compute $x_{j-1}(t)$ at N different values of t , and to compute each of those, we need to compute $x_{j-2}(t)$ at N different values of t . To avoid this, we will use an *interpolation*. The idea is to devise a procedure to compute x_1 once and for all, for all t , so that when we compute $x_2(t)$, we can use the known function $x_1(t)$ rather than having to recompute it over and over again. We will do this in a rather unsophisticated way, but it is possible to do a lot better with more complicated methods.

We will use a *linear interpolation*. The idea is to choose a finite family of t_n 's and record the value of $x_j(t_n)$ for each t_n . To obtain values $x_j(t)$ for $t \neq t_n$, we approximate x_j by a function that is linear between the $x_j(t_n)$. More specifically, let us fix a final time t_F , and define

$$T_n := t_0 + (t_F - t_0) \frac{n}{N} \quad (10)$$

as well as the vector

$$\mathbf{T} = (T_0, T_1, T_2, \dots, T_N). \quad (11)$$

We then define $x_1(t)$ in the following way: let

$$x_1(T_k) = x_0 + \frac{T_k - T_0}{N} \sum_{n=0}^N v(x_0, T_0 + (T_k - T_0) \frac{n}{N}) \quad (12)$$

we then define the vector

$$\mathbf{X}_1 = (x_1(T_0), x_1(T_1), x_1(T_2), \dots, x_1(T_N)). \quad (13)$$

To compute $x_1(t)$, we search through the vector $\mathbf{T}$ for the T_n that is such that $t \in [T_n, T_{n+1})$, and approximate

$$x_1(t) \approx x_1(T_n) + \frac{t - T_n}{T_{n+1} - T_n}(x_1(T_{n+1}) - x_1(T_n)). \quad (14)$$

We can then define $x_2(T_n)$ using this approximation of x_1 , and compute $\mathbf{X}_2$. Proceeding in this way, we compute the Picard sequence $\mathbf{X}_j$.

Exercise 4

Write a routine `interpolate`. The arguments are a number t and two vectors $\mathbf{Y} = (Y_0, \dots, Y_N)$ and $\mathbf{T} = (T_0, \dots, T_N)$ (vectors are represented by arrays in most programming languages). The routine should return

$$Y_n + \frac{t - T_n}{T_{n+1} - T_n}(Y_{n+1} - Y_n) \quad (15)$$

where n is the unique integer such that $t \in [T_n, T_{n+1})$.

Exercise 5

Write a routine `integrate_riemann_interpolated`. This routine should take the following arguments: a function v , a vector $\mathbf{Y}$, two numbers $t_0, t \in \mathbb{R}$, an integer $N \in \mathbb{N}$, and a vector $\mathbf{T}$. The routine then returns

$$\frac{t - t_0}{N} \sum_{n=0}^{N-1} v(x(t_0 + (t - t_0)\frac{n}{N}), (t_0 + (t - t_0)\frac{n}{N})) \quad (16)$$

where $x(t)$ is computed from $\mathbf{Y}$ using the `interpolate` routine.

Exercise 6

Write a routine `picard_interpolated`. This routine should take the following arguments: a function v , a vector $\mathbf{Y}$, three numbers $x_0, t_0 \in \mathbb{R}$, an integer $N \in \mathbb{N}$, and a vector $\mathbf{T} = (T_0, \dots, T_N)$. The routine then returns a vector $\mathbf{X} = (X_0, \dots, X_N)$ with

$$X_i = x_0 + \int_{t_0}^{T_i} v(y(s), s) \quad (17)$$

where the integral is approximated using `integrate_riemann_interpolated`, and $y(s)$ is computed from $\mathbf{Y}$ using `interpolate` (the idea here is that $\mathbf{Y}$ is the previous Picard iterate, and $\mathbf{X}$ is the next Picard iterate: $\mathbf{Y} = \mathbf{X}_n$, $\mathbf{X} = \mathbf{X}_{n+1}$).

Exercise 7

Use `picard_interpolated` to compute the first 10 Picard iterates for equation (8). Take $t_0 = 0$, $N = 100$, $t_F = 2\pi$. On the same graph, plot $x_0(t)$, $x_5(t)$, $x_{10}(t)$ and the exact solution $e^{\sin(t)}$. Note that there is a visible numerical error, which is due to the interpolation and the Riemann sum. This error can be reduced by increasing N : repeat the previous computation for $N = 1000$ and plot your result on a separate graph.

Concluding remarks

As we have seen, in order to get a decent precision, we had to take N to very large values. This is due to the fact that the Riemann sum and the linear interpolation are very poor approximations. A more sophisticated algorithm could use a much more accurate interpolation scheme, such as one based on Chebyshev polynomials. To compute integrals, it is much more efficient to use

Gauss quadratures, which are exponentially accurate for functions that are regular enough. In typical applications, Gauss quadratures and Chebyshev polynomial expansions would allow us to take N as small as 20, and still do better than the algorithm above with $N = 1000$.

We have used the Picard iteration to solve a differential equation in one dimension. The Picard iteration is not the best tool in one dimension: it requires the computation of many integrals, but we have seen that, in one dimension, Barrow's formula will give us the solution with a single integral. So as long as we are willing to approximate integrals numerically, we are better off using Barrow's formula. This procedure is only really useful in higher dimensions. If you wish to try this yourself, everything that has been discussed above extends in an obvious way to higher dimensions.